

LeetCode 1020: Number of Enclaves

Authored by: Arpit Raj | LNMIIT Jaipur | July 22, 2026

Concept Interpretation

Instead of checking each piece of land to see if there is an escape path, the optimal approach works backwards. By scanning the boundary cells and launching a Search (DFS/BFS) from any land found on the edge, we can mark all land cells connected to the outside world.

"ab jo bach gaye wo saare enclave cells he" — Any '1's remaining on the grid are completely surrounded enclaves and cannot escape. We simply count them up.

Approach 1: Depth-First Search (DFS)

```
class Solution {
public:
    vector<pair<int, int>> dir = {{-1, 0}, {1, 0}, {0, 1}, {0, -1}};
    int n;
    int m;

    void dfs(int r, int c, vector<vector<int>>& grid) {

        grid[r][c] = 0;
        for (auto it : dir) {
            int nr = r + it.first;
            int nc = c + it.second;
            if (nr >= 0 && nc >= 0 && nr < n && nc < m) {
                if (grid[nr][nc] == 1) {
                    dfs(nr, nc, grid);
                }
            }
        }
    }

    int numEnclaves(vector<vector<int>>& grid) {

        // concept ill use is that instead of stepping on every land and seeing
        // if it can escape i can simply start from every piece which is 1 and
        // on boundaries and mark all connected lands

        int ans = 0;
        n = grid.size();
        m = grid[0].size();
    }
};
```

```

    for (int i = 0; i < n; i++) { // scanning all leftmost rightmost boundaries if land

        if (grid[i][0] == 1) {
            dfs(i, 0, grid);
        }

        if (grid[i][m - 1] == 1) {
            dfs(i, m - 1, grid);
        }
    }

    for (int j = 0; j < m; j++) {
        if (grid[0][j] == 1) {
            dfs(0, j, grid);
        }

        if (grid[n-1][j] == 1) {
            dfs(n-1, j, grid);
        }
    }
    // ab jo bach gaye wo saare enclave cells he
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < m; j++) {
            if (grid[i][j] == 1)
                ans++;
        }
    }

    return ans;
}
};

```

Approach 2: Breadth-First Search (BFS)

```

class Solution {
public:
    int numEnclaves(vector<vector<int>>& grid) {

        int n=grid.size();
        int m=grid[0].size();

        vector<pair<int,int>> dir={{1,0},{-1,0},{0,1},{0,-1}};

        queue<pair<int,int>> q;
    }
};

```

```

//left right boundary
for(int i=0;i<n;i++){

    if(grid[i][0]==1){

        grid[i][0]=0;
        q.push({i,0});

    }

    if(grid[i][m-1]==1){

        grid[i][m-1]=0;
        q.push({i,m-1});

    }

}

//top bottom boundary
for(int j=0;j<m;j++){

    if(grid[0][j]==1){

        grid[0][j]=0;
        q.push({0,j});

    }

    if(grid[n-1][j]==1){

        grid[n-1][j]=0;
        q.push({n-1,j});

    }

}

while(!q.empty()){

    int r=q.front().first;
    int c=q.front().second;
    q.pop();

    for(auto it:dir){

        int nr=r+it.first;
        int nc=c+it.second;

        if(nr>=0 && nr<n && nc>=0 && nc<m){

```

```
        if(grid[nr][nc]==1){

            grid[nr][nc]=0;

            q.push({nr,nc});

        }

    }

}

int ans=0;

for(int i=0;i<n;i++){

    for(int j=0;j<m;j++){

        if(grid[i][j]==1){

            ans++;

        }

    }

}

return ans;

};
```